Aplicación del Código de Hamming (7,4)

Pérez Leguizamón Andrés Francois 1401638
est.andresf.perezl@unimilitar.edu.co
Docente: Jose Rugeles

***Resumen* — En esta práctica se codificaron dos palabras de datos de 4 bits (Word A = 1011 y Word B = 1001) usando el código de Hamming (7,4), siguiendo el procedimiento visto en clase: se determinó el número de bits de paridad necesarios, se ubicaron los datos en la palabra de 7 bits, se calculó cada bit de paridad con operaciones XOR y se comprobó el resultado. Se obtuvieron las palabras codificadas 1010101 y 1001100. Además, se dañó un bit de cada palabra y se repitió la comprobación para verificar que el error podía localizarse y corregirse.**

***Abstract* — In this practice, two 4-bit data words (Word A = 1011 and Word B = 1001) were encoded using the Hamming (7,4) code, following the procedure shown in class: the number of parity bits needed was determined, the data bits were placed in the 7-bit word, each parity bit was computed with XOR operations, and the result was checked. The resulting codewords were 1010101 and 1001100. In addition, one bit of each codeword was flipped and the check was repeated to verify that the error could be located and corrected.**

***Palabras clave* — Bits de paridad, Código de Hamming, Corrección de errores, XOR.**

***Keywords* — Error correction, Hamming code, Parity bits, XOR.**

I. INTRODUCCIÓN

Cuando se envía información por un canal de comunicaciones, el ruido o las interferencias pueden dañar uno o varios bits del mensaje. El código de Hamming (7,4) es una forma de protegerse de eso: a cada grupo de 4 bits de datos se le agregan 3 bits de paridad calculados con XOR, de modo que si un bit se daña en el camino, el receptor puede darse cuenta y corregirlo sin tener que pedir que se reenvíe el mensaje.

En esta práctica se codificaron las dos palabras de 4 bits asignadas (Word A y Word B) siguiendo el procedimiento visto en clase: primero se determina cuántos bits de paridad se necesitan, luego se ubican los datos y los espacios de paridad en la palabra de 7 bits, después se calcula cada bit de paridad con XOR y, por último, se comprueba que la palabra quedó bien armada y que un error de un solo bit puede corregirse.

II. MARCO TEÓRICO

A. ¿Cuántos bits de paridad se necesitan?

El código (7,4) toma k = 4 bits de datos y les agrega m bits de paridad, para un total de n = k + m bits. Con m bits de paridad se pueden identificar hasta 2^m situaciones distintas (sin error, o error en cualquiera de las n posiciones). Por eso m debe cumplir:

$$2^m \geq k + m + 1$$

Recordando las potencias de 2:

$$2^0 = 1 \qquad 2^1 = 2 \qquad 2^2 = 4 \qquad 2^3 = 8$$

Con m = 3 se tiene $2^3 = 8$, y k + m + 1 = 4 + 3 + 1 = 8, así que 3 bits de paridad alcanzan justo para proteger 4 bits de datos. De ahí sale el nombre "Hamming (7,4)": n = 7 bits en total, de los cuales k = 4 son de datos.

B. ¿En qué posición va cada bit?

La palabra de 7 bits se numera de la posición 1 a la 7. Las posiciones que son potencias de 2 (1, 2 y 4) se dejan para los bits de paridad P_0, P_1 y P_2; las demás posiciones (3, 5, 6 y 7) se llenan con los bits de datos D_4, D_3, D_2 y D_1, en ese orden:

Posición:	7	6	5	4	3	2	1
Bit:	D_1	D_2	D_3	P_2	D_4	P_1	P_0

C. ¿Qué posiciones cubre cada bit de paridad?

Para saber qué posiciones controla cada bit de paridad, se escribe cada posición (0 a 7) en binario de 3 bits (Tabla I). El bit P_0 corresponde al bit menos significativo de esa numeración, P_1 al del medio y P_2 al más significativo.

Tabla I. Posiciones en binario (P_2 P_1 P_0).

Posición	P_2	P_1	P_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

P_0 controla las posiciones donde la columna P_0 vale 1: 1, 3, 5 y 7.
P_1 controla las posiciones donde la columna P_1 vale 1: 2, 3, 6 y 7.
P_2 controla las posiciones donde la columna P_2 vale 1: 4, 5, 6 y 7.

D. ¿Cómo se calcula cada bit de paridad?

Cada bit de paridad se calcula sumando en módulo 2 (XOR) los demás bits de su grupo, de manera que el grupo completo (incluyendo la propia paridad) quede en paridad par, es decir, con una cantidad par de unos. La Tabla II recuerda cómo funciona el XOR, que es la única operación que se usa en todo el proceso.

Tabla II. Tabla de verdad del XOR.

A	*B*	*XOR*	*S*
0	0	⊕	0
0	1	⊕	1
1	0	⊕	1
1	1	⊕	0

III. DESARROLLO DE LA PRÁCTICA

Se asignaron dos palabras de 4 bits (código 1401638):

- Word A: 1 0 1 1
- Word B: 1 0 0 1

Con cada palabra se siguieron los mismos pasos vistos en los videos del profesor:

1. Colocar los datos: se ubican los 4 bits de la palabra en las posiciones 7, 6, 5 y 3, dejando vacías (por ahora) las posiciones 4, 2 y 1 para la paridad.
2. Hallar las posiciones: con la Tabla I se confirma qué posiciones debe revisar cada P_0, P_1 y P_2.
3. Hallar P_0, P_1 y P_2: se hace XOR de los bits de datos que le corresponden a cada paridad (Tabla II) y se anota el resultado en su casilla.
4. Armar la palabra codificada: se juntan los 7 bits ya completos, de la posición 7 a la 1.
5. Comprobación: se vuelve a sumar cada grupo, esta vez incluyendo el bit de paridad ya calculado; si todo quedó bien, el resultado debe dar 0 en los tres grupos.

IV. RESULTADOS

A. Word A (1011)

Paso 1 — Colocar los datos: $D_1D_2D_3D_4$ = 1011 va en las posiciones 7, 6, 5 y 3. Las posiciones 4, 2 y 1 quedan pendientes de paridad:

7	*6*	*5*	*4*	*3*	*2*	*1*
1	0	1	P_2	1	P_1	P_0

Paso 2 — Hallar P_0, P_1 y P_2 (Tabla I + XOR):

P_0 → posiciones 1, 3, 5, 7 → bits conocidos 1, 1, 1 → $1 \oplus 1 \oplus 1 = 1 \rightarrow P_0 = 1$.

P_1 → posiciones 2, 3, 6, 7 → bits conocidos 1, 0, 1 → $1 \oplus 0 \oplus 1 = 0 \rightarrow P_1 = 0$.

P_2 → posiciones 4, 5, 6, 7 → bits conocidos 1, 0, 1 → $1 \oplus 0 \oplus 1 = 0 \rightarrow P_2 = 0$.

Paso 3 — Palabra codificada:

7	*6*	*5*	*4*	*3*	*2*	*1*
1	0	1	0	1	0	1

La Word A queda codificada como **1010101**.

B. Word B (1001)

Paso 1 — Colocar los datos: $D_1D_2D_3D_4$ = 1001 va en las posiciones 7, 6, 5 y 3:

7	*6*	*5*	*4*	*3*	*2*	*1*
1	0	0	P_2	1	P_1	P_0

Paso 2 — Hallar P_0, P_1 y P_2:

P_0 → posiciones 1, 3, 5, 7 → bits conocidos 1, 0, 1 → $1 \oplus 0 \oplus 1 = 0 \rightarrow P_0 = 0$.

P_1 → posiciones 2, 3, 6, 7 → bits conocidos 1, 0, 1 → $1 \oplus 0 \oplus 1 = 0 \rightarrow P_1 = 0$.

P_2 → posiciones 4, 5, 6, 7 → bits conocidos 1, 0, 0 → $1 \oplus 0 \oplus 0 = 1 \rightarrow P_2 = 1$.

Paso 3 — Palabra codificada:

7	*6*	*5*	*4*	*3*	*2*	*1*
1	0	0	1	1	0	0

La Word B queda codificada como **1001100**.

Tabla III. Resumen de la codificación de Hamming (7,4).

Word	*Datos* $D_1D_2D_3D_4$	P_0	P_1	P_2	*Palabra codificada.*
A	1011	1	0	0	1010101
B	1001	0	0	1	1001100

C. Comprobación

Para comprobar que las dos palabras quedaron bien armadas, se vuelve a sumar cada grupo de control, ahora incluyendo el

bit de paridad ya calculado. Si el resultado es 0 en los tres grupos, la palabra es correcta.

Word A (1010101):

- $P_0(1,3,5,7)=1\oplus1\oplus1\oplus1=0$ ✓
- $P_1(2,3,6,7)=0\oplus1\oplus0\oplus1=0$ ✓
- $P_2(4,5,6,7)=0\oplus1\oplus0\oplus1=0$ ✓

Word B (1001100):

- $P_0(1,3,5,7)=0\oplus1\oplus0\oplus1=0$ ✓
- $P_1(2,3,6,7)=0\oplus1\oplus0\oplus1=0$ ✓
- $P_2(4,5,6,7)=1\oplus0\oplus0\oplus1=0$ ✓

Los tres grupos dan 0 en ambos casos, así que las palabras codificadas son correctas.

D. Corrección de un error de un bit

Para comprobar que el código realmente corrige errores, se dañó a propósito un bit de cada palabra ya codificada y se repitió la comprobación del punto anterior sobre la palabra recibida (dañada). Esta vez la suma de los tres grupos ya no da 0, sino un número que, buscado en la Tabla I, indica justo la posición del bit que se dañó.

En la Word A se cambió el bit de la posición 5 (se recibe 1000101). Al comprobar los tres grupos se obtiene s_2,s_1,s_0 = 1,0,1, que en la Tabla I corresponde a la posición 5: ahí está el error. Se invierte ese bit y se recupera la palabra original 1010101. Con la Word B se dañó la posición 3 (se recibe 1001000) y la comprobación da s_2,s_1,s_0 = 0,1,1, es decir, posición 3, recuperando 1001100.

Tabla IV. Corrección de un bit alterado.

Caso	Recibida	s_2,s_1,s_0	Pos. error	Corregida
A, sin error	1010101	000	—	1010101
A, error en pos. 5	1000101	101 = 5	5	1010101
B, error en pos. 3	1001000	011 = 3	3	1001100

Como el código solo agrega 3 bits de paridad, esta comprobación únicamente garantiza corregir un error por palabra; si se dañaran dos bits a la vez, el resultado señalaría una posición equivocada.

V. CONCLUSIONES

Siguiendo el mismo procedimiento visto en clase (colocar los datos, hallar las posiciones con la Tabla I, calcular P_0, P_1 y P_2 con XOR, armar la palabra y comprobar), se codificaron correctamente las dos palabras asignadas: la Word A (1011) quedó como 1010101 y la Word B (1001) quedó como 1001100.

La comprobación final, sumando cada grupo con su propio bit de paridad, dio 0 en los tres grupos para ambas palabras, lo que confirma que la codificación se hizo bien. Además, al dañar un bit de cada palabra y repetir esa misma comprobación, el resultado señaló exactamente la posición del bit alterado, permitiendo corregirlo sin necesidad de retransmitir el mensaje.

En resumen, el código de Hamming (7,4) logra este resultado con muy poca información extra (solo 3 bits de paridad por cada 4 bits de datos), lo que lo hace un método sencillo y eficiente para proteger bloques pequeños de datos frente a errores aislados de un bit.

VI. BIBLIOGRAFIA

[1] J. Rugeles, "Actividad de Codificación de Hamming," Universidad Militar Nueva Granada, Facultad de Ingeniería, 2026.